

Day 11: Structs, Bit-fields & Unions

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Structs for Grouping Related Data

A `struct` groups logically related variables under one name. In embedded, structs are used extensively for:

- **Sensor readings** (timestamp + value + channel + validity flag)
- **Configuration blobs** stored in Flash or EEPROM
- **Protocol packets** (header + payload + CRC)
- **Register mappings** (see bit-fields below)

```
struct sensor_reading {
    uint32_t timestamp_ms;
    int16_t  temperature_cdeg; // centidegrees
    uint16_t humidity_pct;     // 0-10000 = 0.00-100.00%
    uint8_t  sensor_id;
    bool     valid;
};
```

Bit-fields for Register Mapping

Bit-fields let you address individual bits or groups of bits within a `uint32_t` register:

```
struct gpio_pin_cnf {
    uint32_t dir    : 1; // Bit 0
    uint32_t input  : 1; // Bit 1
    uint32_t pull   : 2; // Bits 3:2
    uint32_t drive  : 3; // Bits 10:8
```

```
// ...  
};
```

This maps exactly to the nRF52840 GPIO PIN_CNF register. Setting `cfg.dir = 1` sets bit 0 — the hardware output direction bit.

__packed Attribute

By default, the compiler adds **padding bytes** between struct members to ensure each member is aligned to its natural alignment (e.g., a `uint32_t` at a 4-byte address). This breaks register mapping. Use `__packed` to eliminate padding:

```
struct __packed uart_frame {  
    uint8_t  start_byte;    // offset 0  
    uint16_t length;        // offset 1 (not aligned – only valid because __packed)  
    uint8_t  data[16];      // offset 3  
    uint8_t  crc;           // offset 19  
}; // sizeof = 20 (no padding)
```

Warning: Unaligned accesses on non-packed structs can cause HardFaults on some Cortex-M configurations. Always use `memcpy` to access packed struct fields if in doubt.

Unions for Type Punning

A `union` uses the same memory for multiple types — all members share the same address. This is called **type punning** — reinterpreting the same bytes as different types:

```
union sensor_word {  
    float    as_float;    // 4 bytes  
    uint32_t as_u32;      // same 4 bytes  
    uint8_t  bytes[4];    // same 4 bytes  
};  
// Read as float, inspect raw bytes:  
sensor_word w; w.as_float = 3.14f;  
printf("bytes: %02X %02X %02X %02X", w.bytes[0], w.bytes[1], w.bytes[2], w.bytes[3]);
```

Common use: receiving a 4-byte SPI word that could be a float or a `uint32_t` depending on the command.

Zephyr sys_ Register Structs

Zephyr defines register structs for all nRF52840 peripherals in the Nordic MDK (included via the Zephyr SDK). For example, `NRF_UART_Type` maps the UART peripheral registers.

Practice Tasks

1. Map the full nRF52840 GPIO PIN_CNF register (see Nordic PS) as a bit-field struct and verify with `sizeof`
 2. Create a union that stores a `uint32_t` and access it as `uint8_t bytes[4]` — verify byte order (little-endian on ARM)
 3. Build a UART packet struct with `__packed` and verify `sizeof` matches the expected byte count
 4. Use `offsetof()` macro to find the byte offset of each member in a struct — detect any padding
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day11
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)

- Source: [src/day11/main.cpp](#)